LAPORAN PROJECT APLIKASI TO-DO LIST

Dosen Pengampu : PAHRUL IRFAN S.Kom., M.Kom.

Nama : Amalia Mirasafitri

NIM : F1D02310002

Kleas : C

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIS

UNIVERSITAS MATARAM

2026

1. Deskripsi Aplikasi

Aplikasi To-Do List App merupakan aplikasi manajemen tugas berbasis desktop yang dibuat menggunakan bahasa pemrograman Python dengan library PySide6. Aplikasi ini digunakan untuk membantu pengguna mencatat dan mengelola tugas sehari-hari agar lebih teratur.

Aplikasi menyediakan form input dengan minimal lima field, yaitu judul tugas, deskripsi, kategori, prioritas, dan deadline. Form tambah dan edit ditampilkan dalam dialog terpisah, sedangkan tampilan utama digunakan untuk menampilkan daftar tugas dalam bentuk tabel menggunakan QTableWidget.

Interaksi antar komponen menggunakan signals dan slots, dan tampilan antarmuka disusun menggunakan layout manager seperti QVBoxLayout dan QHBoxLayout agar tampilan rapi. Data tugas disimpan dalam database SQLite, sehingga data tetap tersimpan meskipun aplikasi ditutup, serta mendukung operasi tambah, tampil, ubah, dan hapus data.

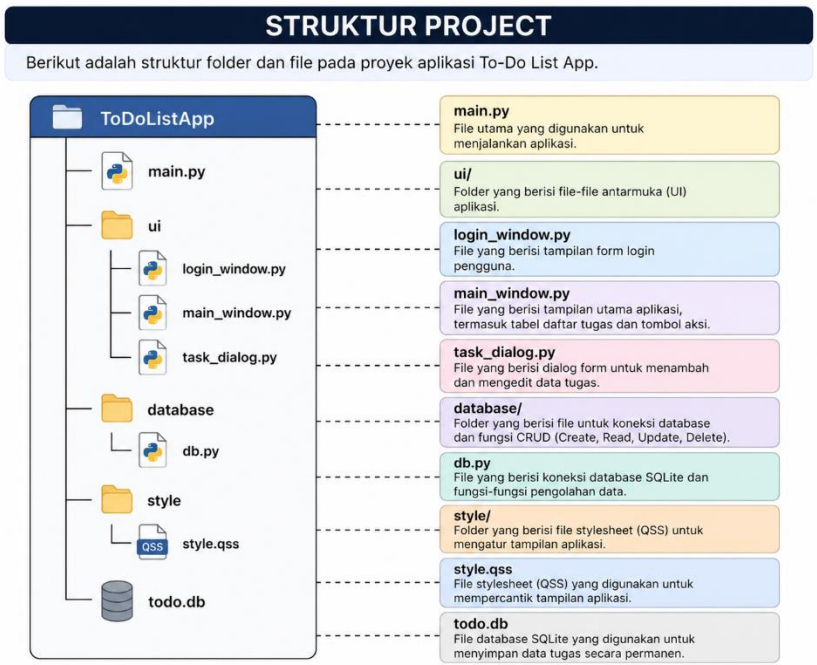
Aplikasi juga memiliki menu bar "About The Application" yang menampilkan informasi nama aplikasi, deskripsi, nama mahasiswa, dan NIM. Selain itu, aplikasi menggunakan QMessageBox sebagai dialog konfirmasi, styling menggunakan QSS dari file eksternal, serta struktur proyek disusun berdasarkan prinsip Separation of Concerns agar kode lebih rapi dan mudah dikelola.

2. Tujuan Project

Tujuan dari pembuatan Aplikasi To-Do List App adalah untuk membantu pengguna dalam mencatat, mengelola, dan memantau tugas sehari-hari agar lebih teratur dan tidak mudah lupa. Aplikasi ini memungkinkan pengguna untuk menambahkan, melihat, mengedit, dan menghapus tugas sehingga pekerjaan dapat dikelola dengan lebih efisien.

Selain itu, project ini bertujuan untuk melatih kemampuan dalam membuat aplikasi desktop menggunakan Python dan PySide6, termasuk penggunaan layout manager, signals dan slots, serta integrasi dengan database SQLite untuk menyimpan data secara permanen. Project ini juga membantu memahami penyusunan struktur program yang rapi dan mudah dikembangkan.

3. Struktur Project



4.

5. Penerapan SoC

Pada aplikasi To-Do List App, prinsip Separation of Concerns (SoC) diterapkan dengan cara memisahkan setiap bagian program berdasarkan fungsi dan tanggung jawabnya. Setiap file dan folder memiliki tugas yang berbeda, sehingga kode menjadi lebih rapi, mudah dipahami, dan lebih mudah diperbaiki jika terjadi kesalahan.

Bagian antarmuka pengguna (UI) ditempatkan dalam folder ui, yang berisi file seperti "`login_window.py`", "`main_window.py`", dan "`task_dialog.py`". File "`login_window.py`" digunakan untuk menampilkan form login pengguna, main_window.py digunakan untuk menampilkan halaman utama yang berisi tabel daftar tugas, sedangkan "`task_dialog.py`" digunakan untuk menampilkan dialog form untuk menambah dan mengedit data tugas. Pemisahan ini membuat tampilan aplikasi tidak tercampur dengan logika database.

Bagian pengolahan data dan database ditempatkan dalam folder database, yaitu pada file "`db.py`". File ini bertugas untuk menghubungkan aplikasi dengan database SQLite serta menjalankan operasi pengolahan data seperti menambah, membaca, mengubah, dan menghapus data tugas. Dengan memisahkan database ke file tersendiri, perubahan pada database tidak akan memengaruhi tampilan antarmuka secara langsung.

Bagian styling atau tampilan desain dipisahkan dalam file "`style.qss`" yang disimpan pada folder style. File ini digunakan untuk mengatur warna, ukuran tombol,

dan tampilan antarmuka aplikasi tanpa harus mengubah kode Python. Hal ini memudahkan perubahan desain tanpa mengganggu fungsi utama aplikasi.

Selain itu, file "`main.py`" digunakan sebagai file utama untuk menjalankan aplikasi dan menghubungkan semua bagian yang telah dipisahkan tersebut. Dengan menerapkan prinsip Separation of Concerns, setiap bagian aplikasi memiliki tanggung jawab yang jelas, sehingga program menjadi lebih terstruktur, mudah dikembangkan, dan mudah dipelihara di masa mendatang.

6. Desain Database

Database pada aplikasi To-Do List App menggunakan SQLite untuk menyimpan seluruh data tugas yang dimasukkan oleh pengguna. Database ini dirancang sederhana dengan satu tabel utama yang digunakan untuk menyimpan informasi tugas secara terstruktur. Data yang disimpan akan tetap tersedia meskipun aplikasi ditutup. Tabel utama yang digunakan dalam database adalah tabel tasks, yang berisi beberapa field untuk menyimpan informasi tugas seperti judul, deskripsi, kategori, prioritas, deadline, dan status tugas.

```
import sqlite3

class Database:
def __init__(self):
    self.conn = sqlite3.connect("todo.db")
    self.create_table()

def create_table(self):
    # tabel task
    self.conn.execute("""
    CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            description TEXT,
            category TEXT,
            priority TEXT,
            deadline TEXT,
            status TEXT DEFAULT 'Belum'
    )
    """)

    # tabel user
    self.conn.execute("""
    CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE,
            password TEXT
    )
    """)
    self.conn.commit()

# Task
def add_task(self, data):
```

```
        self.conn.execute("""
        INSERT INTO tasks (title, description, category, priority,
        deadline)
        VALUES (?, ?, ?, ?, ?)
        """, data)
        self.conn.commit()

    def get_tasks(self):
        return self.conn.execute("SELECT * FROM tasks").fetchall()

    def delete_task(self, task_id):
        self.conn.execute("DELETE FROM tasks WHERE id=?",
        (task_id,))
        self.conn.commit()

    def update_status(self, task_id, status):
        self.conn.execute("UPDATE tasks SET status=? WHERE id=?",
        (status, task_id))
        self.conn.commit()

    # Tambahan untuk update task
    def update_task(self, task_id, data):
        self.conn.execute("""
        UPDATE tasks
        SET title=?, description=?, category=?, priority=?,
        deadline=?
        WHERE id=?
        """, (*data, task_id))
        self.conn.commit()

    # User
    def register_user(self, username, password):
        try:
                self.conn.execute(
                "INSERT INTO users (username, password) VALUES (?,
                ?)",
                (username, password)
                )
                self.conn.commit()
                return True
        except:
                return False

    def login_user(self, username, password):
        cursor = self.conn.cursor()
        cursor.execute(
                "SELECT * FROM users WHERE username=? AND
                password=?",
                (username, password)
        )
        return cursor.fetchone()
```

Kode program di atas digunakan untuk mengelola database pada aplikasi To-Do List App menggunakan SQLite. Database ini berfungsi untuk menyimpan data tugas dan data pengguna agar tetap tersimpan meskipun aplikasi ditutup. Kode dibuat dalam bentuk class Database, yang bertugas untuk menghubungkan aplikasi dengan database serta

menjalankan berbagai operasi pengolahan data seperti menambah, menampilkan, mengubah, dan menghapus data.

Pada bagian awal, program membuat koneksi ke database dengan nama todo.db menggunakan fungsi `sqlite3.connect`. Jika file database belum tersedia, maka SQLite akan membuat database baru secara otomatis. Setelah koneksi berhasil dibuat, program akan memanggil fungsi `create_table()` untuk memastikan bahwa tabel yang dibutuhkan sudah tersedia dalam database.

Fungsi `create_table()` digunakan untuk membuat dua tabel utama, yaitu tabel tasks dan tabel users. Tabel tasks digunakan untuk menyimpan data tugas yang dimasukkan oleh pengguna. Tabel ini memiliki beberapa field, yaitu id sebagai primary key yang bertambah secara otomatis, title untuk menyimpan judul tugas, description untuk menyimpan deskripsi tugas, category untuk menyimpan kategori tugas, priority untuk menyimpan tingkat prioritas tugas, deadline untuk menyimpan batas waktu tugas, serta status untuk menyimpan status tugas dengan nilai awal "Belum". Selain itu, tabel users digunakan untuk menyimpan data pengguna yang dapat login ke dalam aplikasi. Tabel ini terdiri dari id sebagai primary key, username yang bersifat unik agar tidak ada nama pengguna yang sama, serta password untuk menyimpan kata sandi pengguna.

Kode ini juga memiliki beberapa fungsi untuk mengelola data tugas. Fungsi `add_task()` digunakan untuk menambahkan data tugas baru ke dalam tabel tasks. Fungsi `get_tasks()` digunakan untuk mengambil seluruh data tugas dari database untuk ditampilkan pada tabel di antarmuka aplikasi. Fungsi `delete_task()` digunakan untuk menghapus data tugas berdasarkan id, sedangkan fungsi `update_status()` digunakan untuk mengubah status tugas, misalnya dari "Belum" menjadi "Selesai". Selain itu, fungsi `update_task()` digunakan untuk mengubah data tugas yang sudah ada, seperti judul, deskripsi, kategori, prioritas, dan deadline.

Selain mengelola data tugas, kode ini juga menyediakan fungsi untuk mengelola data pengguna. Fungsi `register_user()` digunakan untuk menambahkan pengguna baru ke dalam tabel users. Jika username yang dimasukkan sudah digunakan sebelumnya, maka proses pendaftaran akan gagal. Fungsi `login_user()` digunakan untuk memeriksa kecocokan username dan password yang dimasukkan pengguna dengan data yang tersimpan dalam database. Jika data sesuai, maka pengguna dapat masuk ke dalam aplikasi.

Secara keseluruhan, kode database ini berfungsi sebagai penghubung antara aplikasi dan database SQLite. Dengan adanya kode ini, aplikasi dapat menyimpan, mengambil, mengubah, dan menghapus data secara terstruktur. Hal ini membuat aplikasi menjadi lebih terorganisir dan memastikan bahwa data pengguna dan data tugas dapat dikelola dengan baik.

7. Fitur Utama dan Cara Kerja

Aplikasi ini telah mengimplementasikan fitur CRUD secara lengkap.

a. Fitur *Create* Fungsi `create_table()` digunakan untuk membuat tabel tasks pada database SQLite jika tabel tersebut belum ada. Perintah `CREATE TABLE IF NOT EXISTS` memastikan bahwa tabel hanya dibuat sekali sehingga data yang sudah ada tidak terhapus. Tabel tasks memiliki beberapa field, yaitu id sebagai primary key yang bertambah otomatis (`AUTOINCREMENT`), title untuk judul tugas, description untuk deskripsi, category untuk kategori, priority untuk tingkat prioritas, deadline untuk batas waktu, serta status yang memiliki nilai awal "Belum". Dengan fungsi ini, database siap digunakan untuk menyimpan dan mengelola data tugas dalam aplikasi.

```
def create_table(self):
# tabel task
self.conn.execute("""
CREATE TABLE IF NOT EXISTS tasks (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT,
      description TEXT,
      category TEXT,
      priority TEXT,
      deadline TEXT,
      status TEXT DEFAULT 'Belum'
)
""")
```

b. Fitur *Read* Fungsi `get_tasks()` digunakan untuk mengambil seluruh data tugas dari tabel tasks yang terdapat pada database SQLite. Perintah `SQL SELECT * FROM tasks` berfungsi untuk membaca semua data yang tersimpan dalam tabel tersebut. Hasil dari perintah ini kemudian dikembalikan menggunakan `fetchall()`, sehingga semua data tugas dapat ditampilkan pada tabel di antarmuka aplikasi. Dengan adanya fungsi ini, pengguna dapat melihat daftar tugas yang telah disimpan di dalam database.

```
def get_tasks(self):
    return self.conn.execute("SELECT * FROM tasks").fetchall()
```

c. Fitur *Update* Fungsi edit_`task()` digunakan untuk mengubah data tugas yang sudah ada berdasarkan task_id tertentu. Fungsi ini terlebih dahulu mengambil semua data tugas dari database menggunakan `get_tasks()`, kemudian mencari data yang memiliki id yang sama dengan `task_id`. Jika data ditemukan, maka program akan membuka TaskDialog dan mengisi form dengan data tugas yang lama, seperti judul, deskripsi, kategori, prioritas, dan deadline. Setelah pengguna mengubah data dan menekan tombol simpan, data baru akan diambil menggunakan `get_data()` dan disimpan kembali ke database melalui fungsi `update_task()`. Setelah proses berhasil, akan muncul pesan "Success", dan tabel akan diperbarui kembali menggunakan `load_data()` agar perubahan data langsung terlihat.

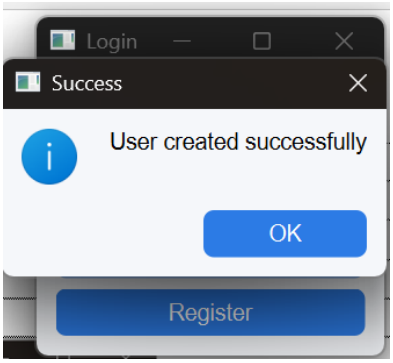
```
def edit_task(self, task_id):
     data = self.db.get_tasks()

     for task in data:
         if task[0] == task_id:
                dialog = TaskDialog(self)

                dialog.title.setText(task[1])
                dialog.desc.setText(task[2])
                dialog.category.setCurrentText(task[3])
                dialog.priority.setCurrentText(task[4])
                dialog.deadline.setText(task[5])

               if dialog.exec():
                  new_data = dialog.get_data()
                  self.db.update_task(task_id, new_data)
                  QMessageBox.information(self, "Success",
     "Updated")
                  self.load_data()
                  break
```

d. Fitur *Delete* Fungsi `delete_task()` digunakan untuk menghapus data tugas dari database berdasarkan task_id tertentu. Pada fungsi ini, program memanggil `self.db.delete_task(task_id)` untuk menjalankan perintah penghapusan data pada tabel tasks sesuai dengan ID yang dipilih. Setelah data berhasil dihapus, fungsi `self.load_data()` dipanggil untuk memuat ulang data dari database dan menampilkan kembali isi tabel, sehingga data yang telah dihapus tidak lagi muncul pada tampilan aplikasi.

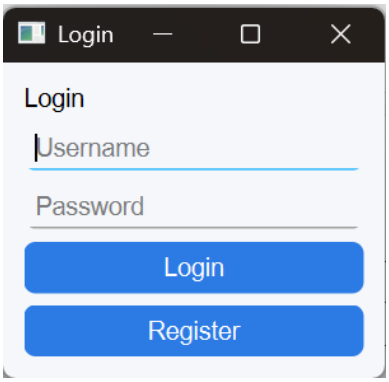
```
def delete_task(self, task_id):
    self.db.delete_task(task_id)
    self.load_data()
```

8. Screenshot Antarmuka

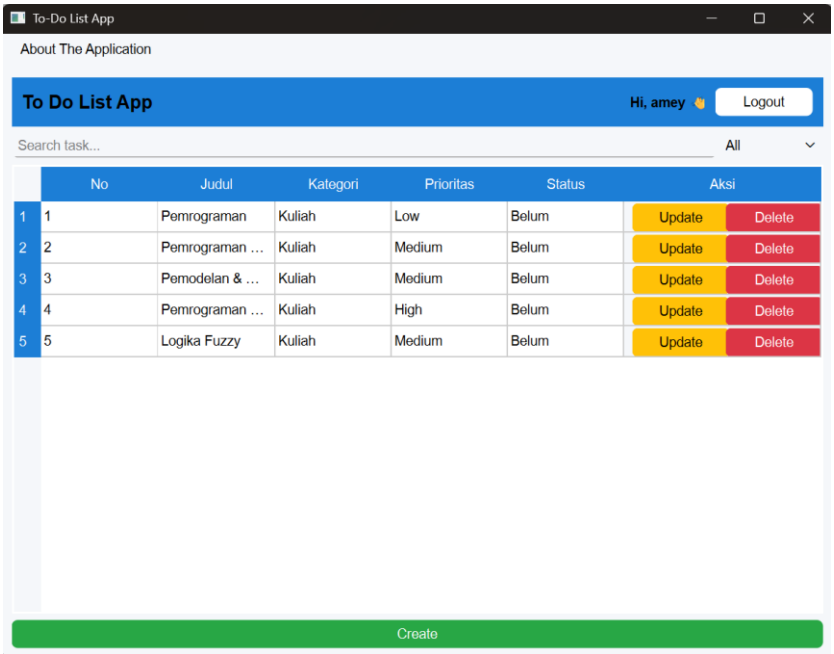
a. Tampilan Register



b. Tampilan Login

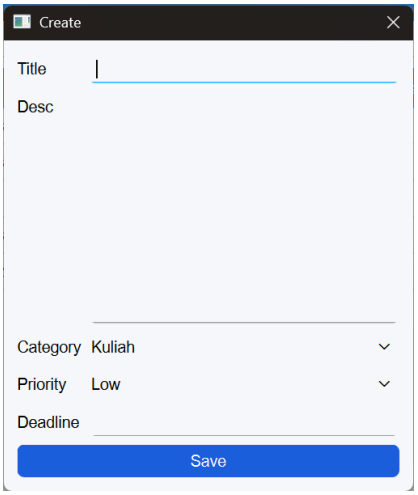


c. Tampilan Utama

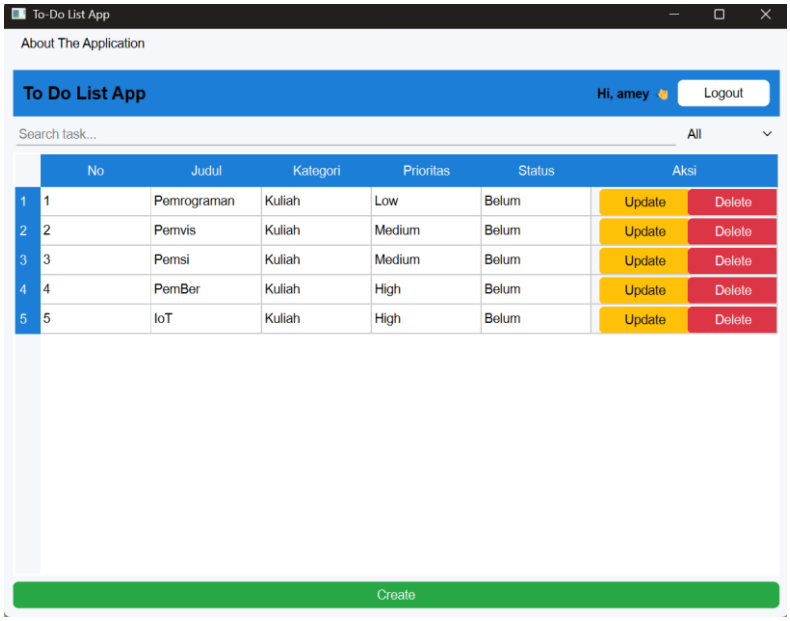


d. Tampilan entang Aplikasi

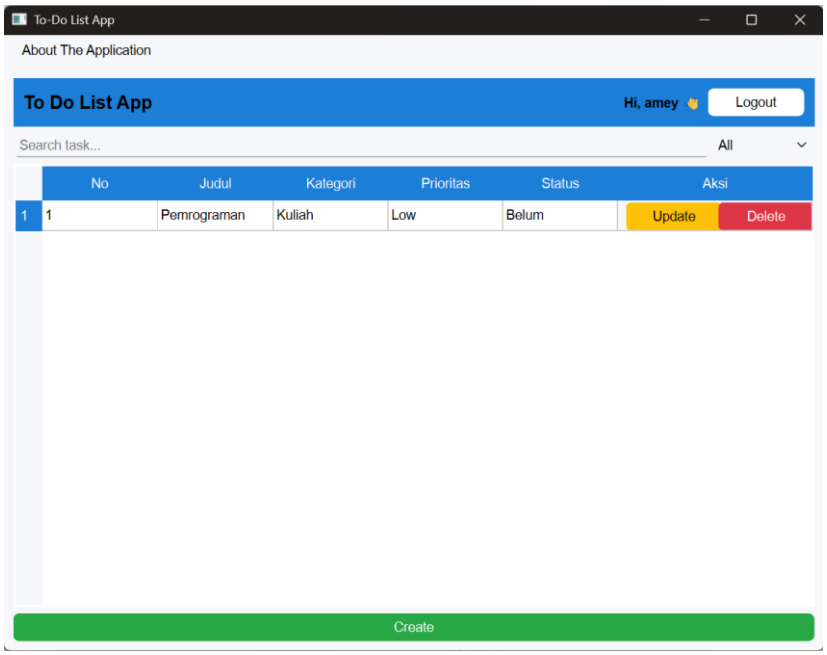
e. Tampilan Create



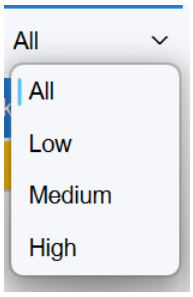
f. Tampilan Update



g. Tampilan Delete



h. Tampilan Prioritas



9. Link Youtube

https://youtu.be/zuP9G1CIEH4?si=Qy1xM6SbfzxS0kfp

10. Link Github

https://github.com/amaliamirasafitri/pv26-miniproject-todolist-F1D02310002